

Testing playbook

This tab is the working detail behind section 10 of the main plan: test cases, scripts, the fault table and gate checklists. Targets marked (*team target*) are goals the team chose, not industry rules. Tool facts were checked on official pages on 2–3 Oct 2026.

1. Words you will meet

Term	Plain meaning
Unit / integration / E2E test	One function alone / several parts together (API + database) / a robot browser clicking through the real app
pgTAP	Tests that run <i>inside</i> PostgreSQL; used for RLS rules, the ledger and the contact filter (Supabase guide)
Smoke test	A 5-minute "is it on fire?" check after each deploy
p95	Sort 100 response times and take the 95th. "p95 = 400 ms" means 95 of 100 requests took 400 ms or less
SLO / error budget	The reliability target (99.5% up) / what is left over (0.5%, about 3.6 h a month)
Idempotent	Doing it twice has the same effect as doing it once
VU	A virtual user in a k6 load test
Load / stress / soak	Expected traffic / traffic pushed until it breaks / normal traffic for many hours
RPO / RTO	How much data you could lose in a disaster / how long recovery may take

2. Environments, commands and CI

Test data rules:

- Never copy production data to staging or a laptop.

- Staging uses fake data only, **including fake identity documents**.
- Each role has its own staging test accounts: student A and B, mentor DE and UK, ops-admin, money-admin with 2FA, and (from launch) guardian.
- Passwords live in the team password manager.

When	What runs (any failure blocks the PR)
Before every push	<code>pnpm test:fast : lint + types + unit tests for web and API</code>
Every PR	ESLint, <code>tsc</code> , Ruff, mypy; Vitest; <code>uv run pytest -m "not integration and not eval"</code> with the coverage gate (90% line / 85% branch on <code>apps/api/app/domain/</code>); bundle-size budget; gitleaks; dependency audit; test-guard ; sensitive-pr
PRs touching <code>supabase/</code> or <code>apps/api/</code>	<code>supabase start && supabase test db (pgTAP)</code> → <code>uv run pytest -m integration</code> → the access test that logs in as every role
After a preview deploy	Playwright smoke (5 journeys) + axe
Nightly, 02:00 PKT, on staging	Full Playwright; <code>supabase db advisors</code> ; k6 1-user smoke; Schemathesis (from S6); backup job
Weekly (Sunday)	Lighthouse on 5 pages (FE); top-10 slowest queries from <code>pg_stat_statements</code>

Test-guard fails a PR if any of these happen:

- `skip`, `xfail`, `.only` or `fixme` is added without a `QUARANTINE(#issue)` note
- a test file is deleted without your approval
- domain coverage drops below `main`

Flaky tests (tests that sometimes fail with no code change) are quarantined within 24 h and fixed within 7 days. E2E tests may retry once; unit and database tests never retry.

3. Daily and feature-wise routine

When	What
Start of a session	<code>git pull</code> , then <code>pnpm test:fast</code> . Start from green
While coding	Write the test with the code. For money, booking and RLS: tests first , from the acceptance criteria
After deploy to staging	Click the happy path on your own phone; once a week over mobile data
End of a session	Glance at Sentry for new errors in your area
Within 48 h of a PR	A test buddy (never the author) spends 20–30 min: key cases on a real Android (iPhone for mentor screens) + 10 minutes of wrong inputs, back button, double taps and slow network. PASS/FAIL comment on the PR
Sunday 20:00–21:00	Test bash : R&D picks a focus area (booking/payment → mentor onboarding → admin → slow network and accessibility) and gives each person a 40-min charter

Manual test case template (key flows only, in `docs/testing/<flow>.md`):

```
ID: TC-<FLOW>-<nn>    Covers: #<issue> AC-<n>    Priority: P1
Preconditions: <test account, flags, data>
Steps: 1. ... 2. ... 3. ...
Expected: <exact result: status, PKR amounts, emails, times>
Device/network: <e.g. Android Chrome, "Slow 4G">
Result log: <date | tester | build | PASS/FAIL | bug link>
```

4. Worked example: booking request + held payment

Example: a 60-min gig at **PKR 5,000**.

- The student pays **PKR 5,250**: the price plus a 5% fee of PKR 250.
- The mentor receives **PKR 4,250**: the price minus 15% commission (PKR 750).
- The platform keeps PKR 1,000.

Money is stored in integer paisa (525,000).

Acceptance criteria (beta)

AC	Given / When / Then
AC-1	A Karachi student sees proposed times in their own time and the mentor's (Berlin or London), stored in UTC, correct across the clock changes on Sun 25 Oct 2026 and Sun 28 Mar 2027
AC-2	Mentor "accept" fails if the session starts less than 36 h later, or overlaps another booking of that mentor
AC-3	Checkout shows PKR $5,000 + 250 = 5,250$, the refund policy in plain words, and "Admission is never guaranteed"
AC-4	The checkout link stops working 24 h before the session; if unpaid, the booking is cancelled, the slot freed and both users emailed
AC-5	Money-admin (2FA) "mark paid" needs a Safepay reference + amount. A wrong amount or reused reference is refused. Success gives exactly one zero-sum ledger group and a confirmed booking
AC-6	Editing or deleting a ledger row raises an error, even for the table owner
AC-7	Reminder emails at T-24 h and T-1 h in each user's time zone; sent once even if the job runs twice; none for a cancelled booking
AC-8	Cancel 24 h or more before: 100% refund. Under 24 h: 50%. A refund can never exceed the held amount
AC-9	Confirm with a rating, or 72 h after the end → money released. A dispute within 72 h → on hold for an admin. No automatic no-show refund

AC	Given / When / Then
AC-10	Student A cannot read student B's booking, order or ledger rows; ops-admin cannot "mark paid"
AC-11	For every order: paid = refunds + mentor payable + platform revenue (+ gateway fee), to the paisa

Key test cases

ID	AC	What is checked	Level
TC-01	1	A Berlin 18:00 slot shows 21:00 PKT on Tue 20 Oct and 22:00 PKT on Tue 27 Oct	Unit + Playwright time zone
TC-02	2	Accept at start – 35 h 59 min refused; at – 36 h accepted	Unit
TC-03	2	Two overlapping accepts: the second is refused by the database constraint	pgTAP + API
TC-04	3	Fees for every price in the bands (Hypothesis property test): total = price × 1.05; mentor = price × 0.85	Unit
TC-06	5	Mark paid: wrong amount → 422; reused reference → 409; admin without 2FA → 403	API
TC-07	5, 11	Mark paid success: one ledger group, sum 0, booking confirmed, 2 emails queued	API + pgTAP
TC-08	6	UPDATE/DELETE on the ledger raises an error, also as owner	pgTAP
TC-09	7	Reminder job run twice → 1 email per user; cancelled → 0	Job test
TC-10	8	Cancel at – 24 h 00 min 01 s → 100%; at – 23 h 59 min 59 s → 50%	Unit + API
TC-11	9	Auto-confirm at end + 72 h 00 min 01 s, not at 71 h 59 min	Job test
TC-12	10	Access matrix rows for bookings, orders and ledger, for all roles	pgTAP

ID	AC	What is checked	Level
TC-13	all	Full journey on a mid-range Android over "Slow 4G" in the sandbox	Manual
TC-14	launch	"Mark paid" then webhook (or the reverse) = exactly 1 ledger group	API
TC-16	launch	Webhook signature checked against a real Safepay-signed sandbox payload , never one our own code signed	Unit
TC-18	launch	20 parallel holds for one slot → 1 × 201 and 19 × 409	API concurrency

Example (TC-14, pytest):

```

async def test_mark_paid_then_webhook_gives_one_ledger_group(api, db,
money_admin, signed_paid_event):
    order = await db.make_order(total_paisa=525_000) #
    PKR 5,250
    r = await api.mark_paid(order.id, ref="SFPY-123", amount=525_000,
as_user=money_admin)
    assert r.status_code == 200
    r = await api.post_webhook(signed_paid_event(order, ref="SFPY-123")) #
    same payment, second writer
    assert r.status_code == 200 #
    always 200
    await db.run_jobs() #
    the worker applies the event
    assert await db.count_ledger_groups(order.id, kind="payment_captured") ==
1
    assert await db.ledger_sum(order.id) == 0

```

Money decisions you must make by Sat 31 Oct

ID	Question
D1	How long a slot is held during instant booking (15 min?)

ID	Question
D2	Is the 5% student fee refunded on a 24-h-plus cancellation? Who pays Safepay's fee on refunds?
D3	Under 24 h: 50% of the price or of the total? Does the mentor get part of the rest?
D4	Rounding rule for paisa
D5	Definition of a mentor or student no-show; is a mentor paid for a student no-show?
D6	Is "24 h before" measured in exact UTC instants? (Recommended: yes)

5. Usability testing

Method: moderated, remote, think-aloud. The participant talks through their thoughts while doing tasks ([NN/g](#)). One moderator stays neutral; one note-taker records. If someone is stuck for 2 minutes, the moderator gives one hint and the task counts as "failed (with help)".

Round	When	On what	Sessions	Focus
R1	19–22 Oct	Figma prototype	5 students + 2 mentors	Is the offer clear? Do people trust "verified"? Is the price breakdown clear?
R2	1–3 Dec	Alpha on staging (sandbox)	5 students + 2 mentors	Can people request, pay and confirm without help? Do they understand "held" money?
R3	1–4 Feb	Release candidate	2 students + 2 mentors + 2 parents	Calendar booking, intro call and refunds, guardian view

Who takes part:

- **Students:** 18+ and in Pakistan. At least 2 per round prefer Urdu, at least 1 lives outside a big city, and at least 3 use their own Android phone.

- **Mentors:** at least one each in Germany and the UK, and one on an iPhone.
- **Parents (R3):** at least one who is less comfortable with technology.

Where to find them: university societies and Pakistani student associations, not close friends (they are too polite).

Thank-you gift: a PKR 1,000 mobile top-up (*estimate*).

Task scripts (participants use a persona card with fake data, never their own grades or documents):

#	Scenario (read aloud)	Success =
S1	"You are Ayesha from Multan, BS Computer Science, small budget. Find a mentor studying in Germany who can review your Statement of Purpose for under PKR 5,000. How do you know they are genuine?"	Opens a matching mentor and points to the verification badge
S2	"Ask this mentor for a 30-minute call next week after 8 pm Pakistan time. Suggest three times. What time is that for the mentor?"	Request sent; both times understood
S3	"Your mentor accepted. Pay with the test card. Where is your money now?"	Pays; explains that the platform holds it
S4	"The call is in 2 days. Cancel it. How much do you get back?"	Cancels; says 100%
S5	"Join the session, switch to audio only, then confirm and rate it."	Done without help
S6	"Write a note to your mentor and include your WhatsApp number."	Understands the block message; watch if they feel insulted
M2	(Mentor) "Accept one of these times. Your gig costs PKR 6,000: how much will you receive?"	Right time; says PKR 5,100
P1	(Parent) "Link to your daughter's account and check what she booked and paid."	Finds booking, amount and mentor

Each 40-min session: 3 min welcome + consent, 2 min think-aloud practice, 25 min tasks, 5 min SUS questionnaire, 5 min debrief ("What confused you most? Would you pay for this?").

Metrics:

Metric	How	Target (<i>team target</i>)
Task success	Done without help = 1. Report as "4/5", not a percentage	Critical tasks (S2, S3, mentor apply) at least 4/5 at R2
Time on task	Stopwatch; report the median	S2 + S3 together under 10 min by R2
Ease after each task	1-7 scale (typical average 5.3-5.6, MeasuringU)	At least 5.5 on critical tasks
SUS	10 statements, 1-5	At least 68 at R2, 75 at R3

How to compute SUS ([MeasuringU](#)):

1. Odd statements: score = answer – 1.
2. Even statements: score = 5 – answer.
3. Add the 10 scores and multiply by 2.5.

Example: answers 4, 2, 5, 1, 4, 2, 5, 2, 4, 1 give odd 17 + even 17 = 34 × 2.5 = **85**. The average across 500 studies is 68.

Consent (Google Form, English + Urdu, before the session):

- what is recorded
- team-only access
- recording deleted within 90 days
- anonymised quotes only
- the participant may stop at any time and still gets the gift
- "do not type real personal data"

Recordings and raw notes **never** go into an AI tool.

From findings to issues:

1. Within 24 h, the note-taker cleans the notes: one row per problem, with a clip time.

2. A 60-minute synthesis rates each problem Critical / Serious / Minor and counts how many participants hit it.
3. Each problem becomes a GitHub issue labelled `usability`. A Critical problem on a core flow is an S2 bug and is fixed before the next round.

6. Performance testing

Test conditions:

Profile	Speed	Use
Slow 4G + 4× CPU (Lighthouse mobile default)	1.6 Mbps down / 750 Kbps up, 150 ms latency	Main target (Lighthouse)
Pakistan weak signal (custom DevTools profile)	400/200 Kbps, 400 ms, 2% loss	Must still work, slowly (<i>team-defined</i>)
Offline / flaky	Playwright <code>offline</code>	Forms and notes must not lose data

Real devices:

- a mid-range Android (2–3 years old, about 4 GB RAM)
- a low-end Android (2–3 GB RAM), before each gate
- an iPhone with Safari (iOS has 46.88% of mobile users in the UK, [StatCounter](#))

Targets:

What	Target
LCP / INP / CLS (Google's "good" thresholds)	≤ 2.5 s / ≤ 200 ms / ≤ 0.1 (web.dev)
First-load JavaScript	≤ 150 KB gzipped on public pages, ≤ 250 KB on dashboards (<i>team target</i>)
Weekly Lighthouse on 5 pages	Performance ≥ 80 , Total Blocking Time ≤ 300 ms
Mentor search	p95 ≤ 500 ms
Booking request / accept / create checkout	p95 $\leq 1,000$ ms

What	Target
Webhook endpoint	p95 ≤ 1 s, always well inside Safepay's 10-s window
Error rate	< 1% overall, < 0.1% on money endpoints
Beta smoke (Sat 5 Dec)	25 VUs browsing + 5 booking requests/min for 10 min
Launch load (by Sat 6 Feb)	200 VUs + 20 booking requests/min for 15 min; stress until p95 > 2 s; breaking point ≥ 2× launch load

k6 rules:

- Run on staging only, with the fake gateway (`PAYMENT_GATEWAY=stub`). **Never load-test Safepay.**
- Upgrade the staging API to Render Starter during the run.
- Keep the test data under 400 MB (Supabase Free allows 500 MB).

```
// tests/load/core_flows.js
import http from 'k6/http';
import { check, sleep } from 'k6';
export const options = {
  scenarios: {
    search: { executor: 'ramping-vus', exec: 'search',
      stages: [{ duration: '2m', target: 25 }, { duration: '8m',
target: 25 }, { duration: '1m', target: 0 } ] },
    booking: { executor: 'constant-arrival-rate', exec: 'requestBooking',
      rate: 5, timeUnit: '1m', duration: '10m', preAllocatedVUs: 10
},
  },
  thresholds: { // any broken threshold = non-zero
exit = FAIL
    'http_req_failed': ['rate<0.01'],
    'http_req_duration{name:search}': ['p(95)<500'],
    'http_req_duration{name:booking}': ['p(95)<1000'],
  },
};
export function search() {
  const r = http.post(`${__ENV.SB_URL}/rest/v1/rpc/search_gigs`,
    JSON.stringify({ corridor: 'DE', max_price: 500000 }));
```

```

    { headers: { apikey: __ENV.SB_ANON, 'Content-Type': 'application/json' },
      tags: { name: 'search' } });
    check(r, { 'search 200': (x) => x.status === 200 });
    sleep(2 + Math.random() * 3);          // think time, like a real user
  }

```

Database checks:

- Turn on `pg_stat_statements` ([Supabase](#)). Each week, review the top 10 queries; any query averaging over 100 ms gets `EXPLAIN ANALYZE` and probably an index.
- A test counts SQL queries per request: "My bookings" must use at most 5, however many bookings exist.

7. Reliability testing

Allowed downtime	Per month	Per week
99%	7.2 h	1.68 h
99.5% (ours)	3.6 h	50.4 min
99.9%	43.2 min	10.1 min

The team picks 99.5% because four part-time students cannot react at night ([Google SRE](#)).

Error budget policy:

- **More than half the budget used:** the next sprint reserves 30% for reliability work.
- **All of it used:** feature freeze, with only S1/S2 fixes ([SRE Workbook](#)).

Monitoring (one stack: Sentry):

- **One uptime monitor** on `api/readyz`. It fails if the database check takes over 2 s, the worker heartbeat is over 2 min old, or the oldest due job is over 10 min late.
- **Error alerts** on money, auth and booking code.
- **One cron monitor** for reconciliation.
- **An alert** when a job goes "dead".
- **Any money mismatch** is an S1 bug.

Scheduled jobs (reminders, 72-h auto-confirm, link expiry, document deletion; later reconciliation and hold expiry):

- They run on time.
- They catch up after the worker is off for 2 h, running overdue work only once.
- Running two copies at once has no double effect.
- Time rules are tested with an injected `now`.

Backups and restore drills:

- **Targets:** RPO 24 h for the database plus 0 lost payments (Safepay's records rebuild the ledger); RTO 4 h.
- **Nightly encrypted dump** to Cloudflare R2: roles (`--role-only`), schema, data (`--data-only --use-copy`) and **auth users**.

Drill steps (the AI Engineer runs them from `docs/runbooks/restore.md` ; R&D keeps time):

1. Restore into a **new, empty** project, never over production.
2. Point a staging copy of the API at it.
3. Check: row counts match the counts logged at dump time, a seeded test user can log in, one IBAN decrypts, and the ledger is zero-sum.
4. Run the smoke suite.
5. Record the times and compare them with the 4-h target.

Drills run by **Wed 9 Dec** and **Wed 10 Feb**, then monthly.

Soak test:

- **When:** Sun 7 Feb night → Wed 10 Feb, about 72 h, at about 20 VUs with the fake gateway.
- **Where:** an always-on team computer, because GitHub jobs stop after 6 h.
- **Pass:** memory growth under 10% after hour 1, errors under 0.5%, p95 at hour 48 within 20% of hour 1, 0 missed jobs, 0 stuck orders.

Rollback:

- **Web:** Vercel Instant Rollback. On Hobby this only goes back to the immediately previous deployment ([Vercel](#)).
- **API:** redeploy the previous commit.

- **Database:** only "expand then contract" migrations (add the new column first, remove the old one in a later release), so a code rollback never needs a database rollback.
- **Kill switches:** pause new bookings; "mark paid" as the webhook fallback.
- **Practice:** on staging in W7, on production in W20. **Target: 15 min or less.**

Game days (Sun 6 Dec and Sun 31 Jan, 20:00–22:00):

- You inject 4 faults from the table below on staging **without saying which**.
- The QA-on-duty person must spot each one from alerts alone and follow the runbook.
- Every gap becomes an issue.

8. Fault-tolerance scenarios

How to inject faults:

- **Fault switches:** read only when `FAULT_INJECTION_ENABLED=true`, which is allowed only on local and staging. **The app refuses to start in production with it set.**
- **Fake gateway:** a control endpoint that can succeed, fail, time out, delay, duplicate, drop, reorder, send poison events or sign with a bad key.

#	Failure	How to inject	Expected safe behaviour	Pass	Kit
F1	Webhook delayed 30 min	Stub delay	"Payment is being confirmed"; reconciliation or the late webhook confirms it	No order stuck over 24 h; no double ledger	Money gate
F2	Webhook duplicated 3×	Stub duplicate	First applied; others get 200 and change nothing	1 ledger group, 1 email	Money gate
F3	Webhook lost	Stub drop	Reconciliation confirms it	Confirmed within 15 min or at the next daily check	Money gate
F4	Events out of order (refund before payment)	Stub reverse	Both get 200; the early one retries until its predecessor is applied	Correct final state	Money gate

#	Failure	How to inject	Expected safe behaviour	Pass	Kit
F4b	Poison event (unknown order, malformed)	Bad event, then a valid one	Poison → retries → dead + alert; the valid one is applied without delay	Valid order confirmed within 1 job cycle	Money gate
F4c	Forged event / wrong key	Unsigned payload; production started with a sandbox key	401, no change; production refuses to start with the wrong key	No state change	Money gate
F5	Gateway down at checkout	Fault switch	"Payment service is busy. You have not been charged." Retry with the same idempotency key; circuit breaker opens after 5 failures in 60 s	0 unhandled errors; 0 duplicate checkouts	Beta
F6	Gateway slow (15 s)	Fault switch	Times out at 10 s; "checking status"; no blind retry	No double checkout	Beta
F7	"Mark paid" typo or double click	API test	Wrong amount → 422; reused reference → 409; double click → 1 ledger group	TC-06/07	Beta
F8	"Mark paid" + webhook on the same order	Shadow mode, then live	One shared transition under a lock	1 ledger group	Money gate
F9	Video fails	Token-error switch; airplane mode for 20 s	Reconnect, or audio-only, or "technical problem → free reschedule"; no payout before Confirm or 72 h	Rejoin within 30 s on Slow 4G	Beta
F10	Attendance data missing or odd (launch)	Drop/duplicate LiveKit events; 3 reconnects; no heartbeat	Outcome UNKNOWN unless evidence agrees; both asked; dispute queue	0 automatic refunds from missing data	Launch
F11	Database unavailable for 60 s	Local docker pause ; staging error-rate switch	503 + friendly page; no half-written orders; alert; recovers by itself	Back within 60 s; ledger checks green	Beta

#	Failure	How to inject	Expected safe behaviour	Pass	Kit
F12	FastAPI down, Supabase up	Fault switch	Browsing still works; booking, pay and admin buttons show "temporarily unavailable"	No blank pages; uptime alert	Beta
F13	Email provider fails	Fault switch	Booking still succeeds; job retries with backoff, then dead + alert	1 email after recovery, not 8	Beta
F14	Reminder job runs twice / late	Run twice in parallel; worker off 2 h	One email; reminders past the session start are skipped	TC-09	Beta
F15	Payout preparation crashes halfway	Kill the worker mid-run	Unique (mentor, period) rows; nothing doubled	Payout total = sum of released orders	Beta
F16	Double booking (overlapping accepts, double tap)	2 parallel accepts	Database constraint rejects the second	1 booking	Beta
F16b	Slot race (instant booking)	20 parallel requests	Exactly 1 hold	TC-18	Launch
F17	Connection lost while sending a message	Playwright offline	Queued and resent with the same message id; no duplicates	50 messages: 0 lost, 0 duplicated	Launch
F18	Upload fails halfway / wrong file	Offline mid-upload; 30 MB file; <code>.exe</code> renamed <code>.pdf</code>	Clear error + retry; no database row until the file is complete and checked	No orphan files after 10 interrupted uploads	Beta
F19	Worker down 2 h	Worker switched off on staging	Jobs wait; alert; on restart each overdue job runs once	Alert in time; no double effects	Beta
F20	Bad deploy	Deploy a broken build on staging	Smoke fails → rollback	15 min or less	Beta
F21	Supabase outage (provider)	Tabletop exercise at a game day	Status banner; <code>/status</code> updated; users told within 30 min	Runbook followed	Launch

Retry rules ([Google SRE](#)):

- Use random exponential backoff.
- User-facing calls get at most 2-3 quick retries.
- Jobs retry up to 8 times, then go to "dead" with an alert.
- Retry in one layer only.
- **Never blindly retry a payment.**

9. Security, accessibility and devices

Access matrix:

- One file, `supabase/tests/access_matrix.yaml`: role × table/function/bucket/endpoint × action → allow/deny. You write it by hand.
- **pgTAP tests**: the AI Engineer generates them from the matrix only, in a separate session.
- **Catalog test**: logs in as every role, calls every table, function, bucket and endpoint, and fails CI if anything lacks a matrix row.

Contact-filter test vectors (one list, `supabase/tests/fixtures/contact_corpus.csv`):

Must be blocked	Must be allowed
0300.123.4567 , 0300/1234567 , 0 3 0 0 1 2 3 4 5 6 7 , Urdu digits ٠٣٠٠١٢٣٤٥٦٧ , 03001234567 , zero-width spaces, full-width digits, +49 151 23456789 , +44 7700 900123 , ali [at] gmail [dot] com , wa.me/... , t.me/... , insta: @handle , numbers in words	"Free slots: 10 11 12 13 14 15", "Fees were 1500 2000 3000 PKR", "I took CS 101 202 303 404", "Deadlines 15/01/2027 31/05/2027", IELTS band lists

Contact-filter targets: 100% of known tricks blocked; false positives at most 2% on 300+ clean messages. Strikes are given only after a human confirms.

Uploads must reject:

- an `.exe` renamed `.pdf` (type checked from the content)
- double extensions
- an SVG with a script
- files over 10 MB

- path-traversal file names

Uploads must also:

- strip photo location data
- keep signed links expiring after 120 s
- delete files 30 days after the decision

OWASP Top 10:2025 focus ([OWASP](#)):

- broken access control (IDOR on every `{id}` route)
- misconfiguration (public bucket, secret key in the web app)
- injection (scripts in bios and notes)
- insecure design (client-sent prices)
- authentication (admin without 2FA)
- integrity (forged webhooks)
- money and access checks must always fail closed

Accessibility:

- **Standard:** WCAG 2.2 AA. Tap targets at least 24 × 24 px (44 px on main buttons).
- **Automated:** axe on every Playwright journey.
- **Manual (W8 and W18):** keyboard-only booking, TalkBack and VoiceOver, 200% zoom, contrast, errors shown as text and not only colour.

Devices: mid-range Android with Chrome on Slow 4G (every test bash), iPhone Safari, low-end Android (before each gate), Windows laptop. Check widths 360, 390, 768 and 1366 px.

10. Bug report template

```
Title: [Area] what is wrong
Environment: production / staging / local – URL – commit
Device + network: e.g. Android 14, Chrome, mobile data / "Slow 4G"
Role + TEST account id (never passwords, never real user data)
Steps: 1. ... 2. ... 3. ...
Expected: ... Actual: ...
How often: always / x of y / once
Evidence: screenshot (no personal data), Sentry link, request_id, time (PKT)
Money involved? y/n Personal data exposed? y/n Proposed severity: S1-S4
```

Beta users report problems through the in-app "Report a problem" form or support. The QA-on-duty person turns each report into an issue using **booking codes only**.

11. Testing calendar

Weeks	Testing work	Big events
W1-W2	CI with coverage gate, test-guard and sensitive-pr; gitleaks; Sentry + test alert; filter tests first	-
W3-W4	Access matrix → pgTAP → policies → break-it check; time-zone tests before Sun 25 Oct	R1 19-22 Oct; test bash #1; calibration Sat 31 Oct; decisions D1-D6
W5-W6	Booking state machine, overlap constraint, 36-h rule; ledger tests first; F7, F16	G1 Sat 14 Nov
W7-W8	Session and refund tests; F9, F13, F14; rollback practice on staging; IDOR, uploads, 2FA; nightly backups	Alpha + dry run from Sat 28 Nov
W9-W10	Fake gateway + k6 smoke Sat 5 Dec; F5, F6, F11, F12, F15, F19, F20; restore drill #1 Wed 9 Dec; freeze; regression Thu 10	R2; game day #1 Sun 6 Dec; pre-beta gate Sat 12 Dec
W11-W12	Daily production smoke; signature test with a real sandbox payload; shadow mode from Sat 19 Dec; webhook fault suite F1-F4c	Beta opens Mon 14 Dec
W13-W14	Tests for L3-L5 written first; slot race; reconciliation; Schemathesis nightly	Money gate review Sat 9 Jan
W15-W16	Hotfixes and smoke only (finals)	Money gate final Sat 23 Jan
W17-W18	Webhook live Mon 25 Jan; attendance F10; ZAP baseline; manual accessibility; k6 launch load + stress	Game day #2 Sun 31 Jan; R3; freeze Sun 7 Feb; soak starts

Weeks	Testing work	Big events
W19– W20	Soak ends Wed 10; restore drill #2; full regression; production rollback practice	Pre-launch gate Sat 13 Feb
W21– W22	Hypercare: S1 acknowledged within 1 h; daily production smoke + reconciliation	Public launch Mon 22 Feb

12. Claude Code vs Antigravity in testing

Claude Code (MAJOR)	Antigravity (MINOR)	Claude chat	By hand
A test suite from an issue's criteria (fresh session, public interface only); pgTAP tests from the access matrix; the fake gateway + fault switches; k6 scripts; seed data; root-causing a flaky test	One test case; one test row; fixing a Playwright locator; a daily browser-agent click-through (a human watches the recording)	Drafting manual test cases from the criteria; usability scripts and the SUS form; summarising anonymised notes	The webhook signature check; the access matrix; the break-it check; reading every generated test; all usability sessions

Rules for AI-written tests:

1. No real user data, documents, keys or Safepay credentials in any AI tool.
2. Tests come from the acceptance criteria in a fresh session (`/clear`) that reads only the public interface, never the implementation it tests.
3. No real network calls in unit tests. Stub Safepay, LiveKit, email and LLMs.
4. Read every generated test. Watch for tests that copy current (possibly wrong) behaviour, or only test a mock.
5. Never skip, delete or weaken a failing test. AI may not change expected money values; only you change money rules, through a decision record.

- For the payment gateway, agents use only `docs/vendor/safepay.md` and never guess endpoints or algorithms.

Browser-agent rules (Antigravity):

- A human logs in once in the agent's separate browser profile ([Antigravity docs](#)). No password is ever typed into a prompt.
- Use agent-only staging accounts (one student, one mentor). Never admin, never production.
- Allowed URLs: `localhost` and the staging domain only.
- The recording is evidence attached to the PR. It never replaces Playwright.

Example prompt (Claude Code, MAJOR):

`/clear`. Run `gh issue view 42` for the acceptance criteria. Read **only** the public functions (signatures and docstrings) in `apps/api/app/domain/order_state.py`, not their bodies. Write `apps/api/tests/unit/test_order_state.py`: for each criterion, 1 happy path, 1 invalid input and 1 forbidden transition. Pass `now` as a parameter. Money is integer paisa. Add a Hypothesis test that `paid = refunds + mentor_payable + platform_revenue` for any price in the bands. Run `uv run pytest apps/api/tests/unit -q`. Do not change production code or expected values. List any criterion you could not test.

Example prompt (Antigravity, MINOR):

MINOR task, 1 file. Add 10 rows to `supabase/tests/fixtures/contact_corpus.csv` with `expected=allowed`: lists of prices, course codes, two dates in one message, IELTS band lists. Same columns as the existing rows; synthetic text only. Do not touch any other file.

13. Later (after launch)

- Network-fault proxy (Toxiproxy)
- Lighthouse in CI
- Field Web Vitals from real users
- Real-time chat tests
- Retrieval metrics for AI

- Urdu right-to-left visual checks
- BrowserStack real devices (Student Pack)
- ZAP active scan, and a paid penetration test after G2
- Point-in-time recovery
- WhatsApp reminder tests
- Mutation testing
- Quarterly usability rounds